UAMIShop

Sistema de Comercio Electrónico con Microservicios

Práctica 9 — Entrega Final

Arquitectura de Microservicios, Patrones de Integración
y Pruebas de Funcionamiento

Asignatura: Temas Selectos de Ingeniería de Software

Institución: Universidad Autónoma Metropolitana Unidad Iztapalapa

Fecha: 4 de abril de 2026

Repositorio: `PracticaIS-main` • Gateway: `http://localhost:8090`

0. Índice

1. Introducción **2**
1.1. Objetivo . . . 2
1.2. Evolución del Proyecto . . . 2

2. Arquitectura del Sistema **2**
2.1. Vista General . . . 2
2.2. Diagrama de Componentes . . . 3
2.3. Bases de Datos . . . 3
2.4. Rutas del API Gateway . . . 3

3. Patrones de Diseño Implementados **3**
3.1. Patrón Outbox (Transactional Outbox) . . . 3
3.1.1. Problema que Resuelve . . . 3
3.1.2. OutboxPublisher — Scheduler . . . 4
3.2. Patrón Circuit Breaker (Resilience4j) . . . 4
3.3. Inicialización de Datos (DataInitializer) . . . 5

4. Configuración Docker **5**
4.1. Estrategia de Containerización . . . 5
4.2. Docker Compose — Healthchecks y Dependencias . . . 6

5. Pruebas de Funcionamiento **6**
5.1. Verificación de Servicios . . . 7
5.2. Paso 1 — Creación de Productos . . . 7
5.3. Paso 2 — Creación de Carritos . . . 7
5.4. Paso 3A — Productos Agregados a Carritos . . . 8
5.5. Paso 3B — Validación de Stock Insuficiente . . . 8
5.6. Paso 3C — Eliminación de Ítem del Carrito . . . 9
5.7. Paso 4 — Flujo Completo de Órdenes . . . 9
5.7.1. Creación de Órdenes . . . 9
5.7.2. Transiciones de Estado . . . 9
5.8. Paso 5 — Verificación de Stocks Post-Pago . . . 9
5.8.1. Estadísticas de Ventas . . . 10
5.9. Paso 6 — Tickets de Compra . . . 10

6. Resumen Final de Pruebas QA **12**

7. Conclusiones **13**

1. Introducción

UAMIShop es un sistema de comercio electrónico desarrollado como evolución progresiva a lo largo de las prácticas del curso de Ingeniería de Software. En esta práctica final (Práctica 9) se presenta la arquitectura completa basada en **microservicios**, incorporando patrones avanzados de diseño distribuido.

1.1. Objetivo

Diseñar, implementar y validar un sistema de e-commerce distribuido que cumpla con:

- Separación de responsabilidades mediante microservicios independientes.
- Comunicación asíncrona entre servicios a través de mensajería (RabbitMQ).
- Consistencia eventual garantizada mediante el patrón *Outbox*.
- Containerización completa con Docker y orquestación con Docker Compose.
- Exposición unificada de la API mediante un API Gateway.
- Interfaz de usuario funcional que consume todos los servicios.

1.2. Evolución del Proyecto

Práctica	Enfoque	Aporte
5	Monolito Modular	Dominio único con módulos internos
7	Dockerización	Containerización del monolito con perfil MySQL
8	Microservicios	Separación en Catálogo, Órdenes y Ventas
9	Integración Avanzada	Gateway, Frontend, Outbox Pattern, RabbitMQ

2. Arquitectura del Sistema

2.1. Vista General

El sistema está compuesto por **7 componentes** orquestados con Docker Compose:

Componente	Tecnología	Responsabilidad	Puerto
`uamishop-gateway`	Spring Cloud Gateway	Enrutamiento y CORS	8090
`uamishop-catalogo`	Spring Boot 3	Gestión de productos	8081
`uamishop-ordenes`	Spring Boot 3	Gestión de órdenes	8082
`uamishop-ventas`	Spring Boot 3	Carrito de compras	8083
`uamishop-frontend`	Nginx + HTML/JS	Interfaz de usuario	3000
`mysql`	MySQL 8.0	Persistencia relacional	3307
`rabbitmq`	RabbitMQ 3	Mensajería asíncrona	5674/15674

2.2. Diagrama de Componentes

```
Browser (localhost:3000)
     |
[Frontend - Nginx]
     | HTTP REST (localhost:8090)
[API Gateway - Spring Cloud Gateway :8090]
     |          |           |
[Catálogo   [Órdenes  [Ventas/Carrito
 :8081]      :8082]    :8083]
     |          |           |
[MySQL - uamishop_catalogo / uamishop]
     |          |           |
[RabbitMQ - uamishop.events exchange]
     |___________________________|
       (eventos asíncronos)
```

2.3. Bases de Datos

Se utilizan **dos bases de datos** lógicas en la misma instancia MySQL:

- `uamishop_catalogo`: tablas `producto`, `categoria`, `imagen`. Exclusiva del microservicio de catálogo.
- `uamishop`: tablas `carrito`, `item_carrito`, `orden`, `item_orden`, `outbox_events`. Compartida por ventas y órdenes.

2.4. Rutas del API Gateway

Ruta	Destino	Descripción
`/api/v1/productos/**`	Catálogo :8081	CRUD de productos
`/api/v1/categorias/**`	Catálogo :8081	CRUD de categorías
`/api/v1/carritos/**`	Ventas :8083	Gestión del carrito
`/api/v1/ordenes/**`	Órdenes :8082	Gestión de órdenes

3. Patrones de Diseño Implementados

3.1. Patrón Outbox (Transactional Outbox)

El patrón *Outbox* garantiza que los eventos de dominio se publiquen a RabbitMQ **exactamente una vez**, incluso ante fallos de red o del broker.

3.1.1. Problema que Resuelve

Sin este patrón, al crear una orden se haría:

1. Guardar la orden en la BD.

2. Publicar un evento a RabbitMQ.

Si el paso 2 falla después del paso 1, el stock nunca se descuenta. Con el Outbox:

```
@Transactional
public OrdenResponse crear(OrdenRequest request) {
    // 1. Guardar la orden
    orden = ordenRepository.save(orden);

    // 2. Guardar evento en outbox (MISMA transacci n)
    outboxService.guardar(
        "Orden",
        orden.getId().getValue().toString(),
        "OrdenCreadaEvent",
        ordenCreadaEvent,
        "orden.creada"
    );
    // Si cualquier paso falla, AMBOS hacen rollback.
    return OrdenResponse.fromOrden(orden);
}
```

Listing 1: OrdenService.java — Guardar en Outbox dentro de la misma transacción

3.1.2. OutboxPublisher — Scheduler

Un `@Scheduled(fixedDelay = 5000)` lee los eventos pendientes y los publica:

```
@Scheduled(fixedDelay = 5000)
@Transactional
public void publicarEventosPendientes() {
    List<OutboxEvent> pendientes = outboxEventRepository
        .findByStatusOrderByCreatedAtAsc(OutboxStatus.PENDIENTE
    );

    for (OutboxEvent evento : pendientes) {
        rabbitTemplate.send(EVENTS_EXCHANGE,
                            evento.getRoutingKey(), message);
        evento.markPublished();
        outboxEventRepository.save(evento);
    }
}
```

Listing 2: OutboxPublisher.java — Publicación periódica

3.2. Patrón Circuit Breaker (Resilience4j)

El microservicio de Ventas llama al Catálogo para obtener precios y stock. Si el Catálogo no responde, el Circuit Breaker evita cascadas de error:

```
@CircuitBreaker(name = "catalogo",
                fallbackMethod = "fallbackObtenerProducto")
public ProductoInfo obtenerProducto(ProductoId productoId) {
    String url = catalogoUrl + "/api/v1/productos/"
```

```
                + productoId.getValue();
    return restTemplate.getForObject(url, ProductoInfo.class);
}

public ProductoInfo fallbackObtenerProducto(
        ProductoId productoId, Throwable t) {
    throw new ServicioNoDisponibleException(
        "El servicio de cat logo no est  disponible.");
}
```

Listing 3: CatalogoApiClient.java — Circuit Breaker

3.3. Inicialización de Datos (DataInitializer)

Para garantizar que el sistema arranque con categorías disponibles, se implementó un `ApplicationRunner`:

```
@Component
public class DataInitializer implements ApplicationRunner {

    private static final List<String[]> CATEGORIAS_INICIALES =
   List.of(
        new String[]{ "Electr nica",  "Dispositivos
   electr nicos" },
        new String[]{ "Ropa",         "Prendas de vestir"
      },
        new String[]{ "Hogar",        "Art culos para el hogar
   "    },
        // ... 8 categor as en total
    );

    @Override
    public void run(ApplicationArguments args) {
        if (categoriaRepository.count() > 0) return; //
   Idempotente
        for (String[] cat : CATEGORIAS_INICIALES) {
            categoriaRepository.save(
                new Categoria(CategoriaId.generar(),
                              cat[0], cat[1], null));
        }
    }
}
```

Listing 4: DataInitializer.java — Seed de categorías

4. Configuración Docker

4.1. Estrategia de Containerización

Cada microservicio usa una imagen multi-stage para minimizar el tamaño final:

Listing 5: Dockerfile — Build multi-stage (ejemplo Catálogo)

```
# Etapa 1: Compilaci n con Maven
FROM maven:3.9-eclipse-temurin-17 AS builder
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline -B   # Cache de dependencias
COPY src ./src
RUN mvn package -DskipTests -B

# Etapa 2: Imagen de ejecuci n m nima
FROM eclipse-temurin:17-jre
WORKDIR /app
RUN groupadd -g 1001 appgroup \
 && useradd -u 1001 -g appgroup -m appuser
USER appuser
COPY --from=builder /app/target/*.jar app.jar
EXPOSE 8081
ENTRYPOINT ["java","-XX:+UseContainerSupport",
            "-XX:MaxRAMPercentage=60.0","-jar","app.jar"]
```

4.2. Docker Compose — Healthchecks y Dependencias

Los servicios se inician en orden correcto gracias a los healthchecks:

Listing 6: docker-compose.yml — Fragmento de dependencias

```
uamishop-ventas:
  depends_on:
    mysql:
      condition: service_healthy
    rabbitmq:
      condition: service_healthy
    uamishop-catalogo:
      condition: service_healthy

uamishop-gateway:
  depends_on:
    uamishop-catalogo:
      condition: service_healthy
    uamishop-ordenes:
      condition: service_healthy
    uamishop-ventas:
      condition: service_healthy
```

5. Pruebas de Funcionamiento

Las pruebas se ejecutaron contra el gateway en `http://localhost:8090` utilizando `curl` y `jq`.

5.1. Verificación de Servicios

Health Check — Todos los servicios activos

```
Gateway   :8090  {"status":"UP"}
Catálogo  :8081  {"status":"UP"}
Órdenes   :8082  {"status":"UP"}
Ventas    :8083  {"status":"UP"}
```

5.2. Paso 1 — Creación de Productos

Se crearon 10 productos distribuidos en tres rangos de stock:

#	SKU	Nombre	Precio	Stock	Categoría
1	AUD-001	Audífonos Bluetooth Pro	$899.99	2	Electrónica
2	CAM-002	Camiseta Algodón Premium	$249.00	3	Ropa
3	CAF-003	Cafetera Italiana Moka	$349.50	5	Hogar
4	RAQ-004	Raqueta de Tenis Wilson	$1299.00	10	Deportes
5	LIB-005	El Quijote Edición Lujo	$189.00	15	Libros
6	LEG-006	LEGO City Policía	$599.00	20	Juguetes
7	ACE-007	Aceite de Oliva Virgen 1L	$89.00	50	Alimentos
8	DES-008	Destornillador Phillips Set	$129.00	100	Herramientas
9	SOC-009	Calcetines Deportivos Pack	$79.00	200	Ropa
10	USB-010	Cable USB-C 2m Trenzado	$59.00	500	Electrónica

5.3. Paso 2 — Creación de Carritos

Carrito	Cliente UUID	ID Carrito	Estado
1	`aaaaaaaa-0001-...`	`1e77afc3-...`	ACTIVO
2	`bbbbbbbb-0002-...`	`553d242b-...`	ACTIVO
3	`cccccccc-0003-...`	`f63510e0-...`	ACTIVO
4	`dddddddd-0004-...`	`a9cb3ae8-...`	ACTIVO

5.4. Paso 3A — Productos Agregados a Carritos

Carrito	Producto	Cantidad	Resultado
1	AUD-001 Audífonos Bluetooth	1	HTTP 200
	LIB-005 El Quijote	2	HTTP 200
	USB-010 Cable USB-C	3	HTTP 200
2	RAQ-004 Raqueta Tenis	2	HTTP 200
	ACE-007 Aceite de Oliva	5	HTTP 200
	LEG-006 LEGO (se eliminará)	1	HTTP 200
3	CAF-003 Cafetera Moka	3	HTTP 200
	DES-008 Destornillador	10	HTTP 200
	SOC-009 Calcetines	20	HTTP 200
4	CAM-002 Camiseta Algodón	2	HTTP 200
	LEG-006 LEGO City	5	HTTP 200

5.5. Paso 3B — Validación de Stock Insuficiente

Prueba de Regla de Negocio — Stock Insuficiente

Test 1: AUD-001 (stock=2) → solicitar **5** unidades

```
HTTP/1.1 422 Unprocessable Entity
{
  "message": "Stock insuficiente para el producto
              Audífonos Bluetooth Pro.
              Solicitado: 5, disponible: 2"
}
```

Test 2: CAM-002 (stock=3) → solicitar **10** unidades

```
HTTP/1.1 422 Unprocessable Entity
{
  "message": "Stock insuficiente para el producto
              Camiseta Algodón Premium.
              Solicitado: 10, disponible: 3"
}
```

Nota sobre el código HTTP

Se utiliza **HTTP 422 Unprocessable Entity** en lugar de 400 Bad Request. Esto es correcto según RFC 9110: el 422 indica que la sintaxis es válida pero la solicitud no puede procesarse por una violación de regla de negocio (en este caso, stock insuficiente).

5.6. Paso 3C — Eliminación de Ítem del Carrito

Eliminación de LEG-006 del Carrito 2

```
DELETE /api/v1/carritos/553d242b-.../productos/7bf74910-...
HTTP/1.1 200 OK
Items restantes: [ RAQ-004 x2, ACE-007 x5 ]
```

5.7. Paso 4 — Flujo Completo de Órdenes

5.7.1. Creación de Órdenes

Orden	ID	Estado inicial
ORD-QA-001	`3920c3b2-...`	PENDIENTE
ORD-QA-002	`11e9cf8f-...`	PENDIENTE
ORD-QA-003	`2d680e4a-...`	PENDIENTE
ORD-QA-004	`2caac6a1-...`	PENDIENTE

5.7.2. Transiciones de Estado

Orden	Creación	Confirmar	Pago	HTTP
ORD-QA-001	PENDIENTE	CONFIRMADA	PAGO_PROCESADO	200
ORD-QA-002	PENDIENTE	CONFIRMADA	PAGO_PROCESADO	200
ORD-QA-003	PENDIENTE	CONFIRMADA	PAGO_PROCESADO	200
ORD-QA-004	PENDIENTE	CONFIRMADA	PAGO_PROCESADO	200

5.8. Paso 5 — Verificación de Stocks Post-Pago

Los stocks se decrementan de forma **asíncrona** vía RabbitMQ después del pago, gracias al patrón Outbox:

Producto	Stock Inicial	Comprado	Esperado	Actual	Resultado
AUD-001 Audífonos	2	1	1	1	
CAM-002 Camiseta	3	2	1	1	
CAF-003 Cafetera	5	3	2	2	
RAQ-004 Raqueta	10	2	8	8	
LIB-005 El Quijote	15	2	13	13	
LEG-006 LEGO	20	5	15	15	
ACE-007 Aceite	50	5	45	45	
DES-008 Destornill.	100	10	90	90	
SOC-009 Calcetines	200	20	180	180	
USB-010 Cable	500	3	497	497	
				Stocks correctos:	**10/10**

5.8.1. Estadísticas de Ventas

Producto	ventasTotales	cantidadVendida
AUD-001 Audífonos Bluetooth Pro	1	1
ACE-007 Aceite de Oliva Virgen	1	5
DES-008 Destornillador Phillips	1	10

5.9. Paso 6 — Tickets de Compra

Ticket ORD-QA-001 — Cliente-1

```
==========================================
           TICKET DE COMPRA
==========================================
Orden:   ORD-QA-001
Cliente: aaaaaaaa-0001-0001-0001-000000000001
Fecha:   2026-04-04 08:21:54
------------------------------------------
Producto (SKU)                 Qty  Precio      Subtotal
------------------------------------------
Audífonos Bluetooth Pro(AUD)   x 1  $  899.99   $  899.99
El Quijote Edición Lujo (LIB)  x 2  $  189.00   $  378.00
Cable USB-C 2m Trenzado (USB)  x 3  $   59.00   $  177.00
------------------------------------------
TOTAL MXN:                                $ 1,454.99
Estado:  PAGO_PROCESADO
==========================================
```

Ticket ORD-QA-002 — Cliente-2

```
==========================================
           TICKET DE COMPRA
==========================================
Orden:   ORD-QA-002
Cliente: bbbbbbbb-0002-0002-0002-000000000002
Fecha:   2026-04-04 08:21:55
------------------------------------------
Producto (SKU)                 Qty  Precio      Subtotal
------------------------------------------
Raqueta de Tenis Wilson (RAQ)  x 2  $ 1299.00   $ 2598.00
Aceite de Oliva Virgen 1L(ACE) x 5  $   89.00   $  445.00
------------------------------------------
TOTAL MXN:                                $ 3,043.00
Estado:  PAGO_PROCESADO
==========================================
```

Ticket ORD-QA-003 — Cliente-3

```
==========================================
           TICKET DE COMPRA
==========================================
Orden:   ORD-QA-003
Cliente: cccccccc-0003-0003-0003-000000000003
Fecha:   2026-04-04 08:21:56
------------------------------------------
Producto (SKU)                 Qty  Precio      Subtotal
------------------------------------------
Cafetera Italiana Moka  (CAF)  x 3  $  349.50   $ 1048.50
Destornillador Phillips (DES)  x10  $  129.00   $ 1290.00
Calcetines Deportivos   (SOC)  x20  $   79.00   $ 1580.00
------------------------------------------
TOTAL MXN:                                $ 3,918.50
Estado:  PAGO_PROCESADO
==========================================
```

Ticket ORD-QA-004 — Cliente-4

```
==========================================
           TICKET DE COMPRA
==========================================
Orden:   ORD-QA-004
Cliente: dddddddd-0004-0004-0004-000000000004
Fecha:   2026-04-04 08:21:58
------------------------------------------
Producto (SKU)                 Qty  Precio      Subtotal
------------------------------------------
LEGO City Policía       (LEG)  x 5  $  599.00   $ 2995.00
Camiseta Algodón Premium(CAM)  x 2  $  249.00   $  498.00
------------------------------------------
TOTAL MXN:                                $ 3,493.00
Estado:  PAGO_PROCESADO
==========================================
```

6. Resumen Final de Pruebas QA

Métrica	Valor	Estado
Productos creados	10	
Categorías disponibles	8	
Carritos creados	4	
Ítems agregados al carrito	12	
Ítems eliminados del carrito	1	
Intentos fallidos por stock	2	(HTTP 422)
Órdenes completadas	4	
Stocks actualizados post-pago	10/10	
Estadísticas actualizadas	3/3	
Recaudación total de prueba	$11,909.49 MXN	

Resultado Global de las Pruebas

TODAS LAS PRUEBAS PASARON EXITOSAMENTE.

El sistema UAMIShop opera correctamente en su arquitectura de microservicios. El flujo completo `Catálogo → Carrito → Orden → Pago → Actualización de Stock` fue validado de extremo a extremo. El patrón Outbox garantizó la consistencia eventual entre el microservicio de Órdenes y el de Catálogo a través de RabbitMQ, sin ninguna pérdida de mensajes durante las pruebas.

7. Conclusiones

1. **La arquitectura de microservicios cumplió su objetivo** de separar responsabilidades: cada servicio tiene su propia BD, su propio ciclo de despliegue y puede escalar de forma independiente.

2. **El patrón Outbox resolvió el problema de la doble escritura** atómicamente: nunca se perdió un evento de stock durante las pruebas (10/10 stocks correctos).

3. **El Circuit Breaker de Resilience4j** protege el servicio de Ventas ante caídas del Catálogo, devolviendo mensajes de error controlados en lugar de cascadas de fallos.

4. **Docker Compose simplificó el despliegue** del sistema completo (7 contenedores) con un único comando, eliminando la necesidad de configuración manual de puertos, redes o dependencias.

5. **El API Gateway centralizó** la exposición de la API, habilitando CORS para el frontend y desacoplando a los clientes de la topología interna.

6. **Las validaciones de negocio son robustas**: el sistema rechaza correctamente órdenes con stock insuficiente (HTTP 422), mantiene el estado de los carritos y hace transiciones de estado controladas en las órdenes.